# COMPUTACIÓN PARALELA Y DISTRIBUIDA

## Laboratorio 1 — ¿Dónde está el paralelismo?

*Semana 1 — Grupo de Laboratorio 2*

## 1. Propósito

Realizar un primer experimento de computación paralela a partir de un problema de procesamiento de datos. El estudiante deberá identificar qué partes del problema pueden ejecutarse de manera independiente, implementar una versión secuencial y una versión paralela, y comparar experimentalmente su rendimiento.

## 2. Objetivos

- Implementar una solución secuencial para un problema de procesamiento de datos.
- Analizar un algoritmo para identificar oportunidades de paralelismo.
- Implementar una primera versión paralela utilizando Threads.
- Medir el tiempo de ejecución para diferentes cantidades de threads.
- Calcular speedup y eficiencia.
- Analizar las limitaciones del paralelismo obtenido.

## 3. Problema

Se dispone de una colección de N números reales. Para cada elemento se debe calcular una transformación y, posteriormente, obtener la suma de los valores transformados.

$$B[i] = A[i] \times A[i] + 3A[i] + 1$$

$$S = \Sigma\, B[i], \quad i = 0, \ldots, N-1$$

El programa deberá generar o inicializar el vector, calcular la transformación para cada elemento y obtener la suma final. La solución debe producir un resultado correcto y reproducible.

## 4. Parte A — Implementación secuencial

- Genere un vector A de números de tipo double.
- Implemente la transformación indicada.
- Calcule secuencialmente la suma de los elementos transformados.
- Utilice un mecanismo de medición de tiempo apropiado, por ejemplo std::chrono.
- Realice varias ejecuciones para obtener mediciones representativas.

Se recomienda comenzar con N = 10^7 elementos y aumentar el tamaño según las características del equipo.

## 5. Parte B — Análisis del paralelismo

1. ¿Qué cálculos pueden realizarse independientemente para diferentes elementos del vector?

2. ¿Qué información debe combinarse al finalizar el procesamiento?

3. ¿Qué variable o variables podrían convertirse en un recurso compartido?

4. ¿Cómo dividiría el vector entre varios threads?

## 6. Parte C — Implementación con Threads

Implemente una versión paralela utilizando threads. Distribuya entre los threads el procesamiento de los elementos del vector y utilice un mecanismo apropiado para obtener la suma final. En esta primera práctica no es necesario implementar manualmente locks, semáforos u otros mecanismos de sincronización.

## 7. Parte D — Experimento de rendimiento

Ejecute el programa utilizando diferentes cantidades de threads.

| Threads | Tiempo (s) | Speedup | Eficiencia |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 4 | | | |
| 8 | | | |
| 16 | | | |
| 32 | | | |

Speedup: $S_p = T_1 / T_p$

Eficiencia: $E_p = S_p / p$

## 8. Parte E — Influencia del tamaño del problema

Repita el experimento para diferentes tamaños:

- N = 10^6
- N = 10^7
- N = 10^8

Compare cómo cambia el rendimiento cuando aumenta el tamaño del problema. Si no dispone de memoria suficiente, seleccione valores adecuados y registre la decisión.

## 9. Parte F — Distribución del trabajo

Represente gráficamente cómo distribuiría el vector entre cuatro threads. Considere una estrategia por bloques y explique cómo asignaría los elementos a cada thread.

1. ¿La división del trabajo propuesta es equilibrada?

2. ¿Qué ocurre si un thread termina su parte antes que los demás?

3. ¿Existe alguna dependencia entre los cálculos realizados para distintos elementos?

4. ¿Qué costo adicional introduce la ejecución paralela?

## 10. Preguntas de análisis

1. ¿Qué parte del algoritmo puede ejecutarse en paralelo? Explique.

2. ¿Qué diferencia existe entre el trabajo de cada thread y la operación necesaria para obtener la suma final?

3. ¿El speedup obtenido es lineal? Justifique utilizando los resultados.

4. ¿A partir de qué cantidad de threads el rendimiento deja de mejorar significativamente?

5. ¿Qué ocurre cuando se utilizan más threads que núcleos disponibles?

6. ¿El tamaño del problema influye en el speedup? Explique.

7. ¿Qué factores pueden impedir que la aceleración sea proporcional al número de threads?

8. ¿Qué ventajas y desventajas observa al dividir el vector por bloques?

## 11. Entregable

Entregue un informe breve acompañado del código fuente. El informe deberá contener:

- Descripción del problema y estrategia de paralelización.
- Análisis de la descomposición del trabajo.
- Código de las versiones secuencial y paralela.
- Tabla de tiempos, speedup y eficiencia.
- Gráfica de speedup en función del número de threads.
- Resultados para diferentes tamaños del problema.
- Representación de la distribución del trabajo.
- Respuestas a las preguntas de análisis.
- Conclusión sobre las ventajas y limitaciones observadas.

## 12. Consideraciones

- Utilice el mismo equipo y condiciones de ejecución para las comparaciones.
- Realice varias ejecuciones y evite utilizar una única medición.
- Verifique que las versiones secuencial y paralela produzcan resultados equivalentes.
- Registre procesador, número de núcleos y memoria RAM.

- No se espera que los resultados sean idénticos entre diferentes computadores; lo importante es interpretar correctamente el comportamiento observado.

## 13. Idea central

**Antes de paralelizar un programa debemos identificar el trabajo independiente, decidir cómo repartirlo y después medir el resultado.**